מבחן בקורס: עקרונות שפות תכנות, 202-1-2051
מועד: ב
תאריך: 5/8/2025
שמות המרצים: מני אדלר, ירון גונן, יובל פינטר
מיועד לתלמידי: מדעי המחשב והנדסת תוכנה
משך המבחן: שלוש שעות
חומר עזר: אסור

הנחיות כלליות:

- יש לענות על כל השאלות בגיליון התשובות. מומלץ לא לחרוג מן המקום המוקצה.
- אם אינכם יודעים את התשובה, ניתן לכתוב 'לא יודע' ולקבל 20% מהניקוד על הסעיף/השאלה.

שאלה 1:	תחביר וסמנטיקה _____	נק 30
שאלה 2:	מערכת טיפוסים _____	נק 20
שאלה 3:	תכנות פונקציונאלי, CPS, רשימות עצלות	נק 30
שאלה 4:	תכנות לוגי _____	נק 20
סה"כ	_____	נק 100

בהצלחה!

שאלה 1: תחביר וסמנטיקה [30 נקודות]

א. [5 נקודות]

סמנו את הכתובות הלקסיקאליות בביטוי הבא:

```
(lambda (x y)
  (
    (lambda (y z) (+ [y 0 0] [z 0 1] [x 1 0] [a free]))
    [x 0 0] [y 0 1] [z free]
  )
)
```

ב. השפה L20 מוגדרת כשפה L2 אך ללא define

```
<program> ::= (L20 <cexp>+)              / Program(exps:List(cexp))
<var-ref> ::= <identifier>              / VarRef(var:string)
<var-decl> ::= <identifier>    / VarDecl(var:string)
<cexp> ::= <number>                     / NumExp(val:number)
        |  <boolean>                    / BoolExp(val:boolean)
        |  ( lambda ( <var-decl>* ) <cexp>+ ) /
                                ProcExp(args:VarDecl[],body:CExp[]))
        |  ( if <cexp> <cexp> <cexp> ) / IfExp(test: CExp,
        |                                      then: CExp,
        |                                      alt: CExp)
        |  ( <cexp> <cexp>* )               / AppExp(operator:CExp,
        |                                            operands:CExp[]))
<prim-op>  ::= + | - | * | / | < | > | = | not |  eq?
<num-exp>  ::= a number token
<bool-exp> ::= #t | #f
```

הגדירו באופן מפורט (אין צורך בקוד) אלגוריתם להמרת תוכנית ב-L2 לתוכנית ב-L20
[5 נקודות]

בהינתן תוכנית ב-L2:

- **נוסיף כתובות לקסיקאליות ל-CExp ב-AST של התוכנית. כזכור, עבור VarRef של משתנים גלובאליים הכתובת הלקסיקאלית היא free.**
- **נגדיר מבנה נתונים הקושר משתנה גלובלי ל-CExp שלו**
- **נעבור על ה-AST, החל מסביבה ריקה:**
  - **עבור DefExp, נוסיף את המשתנה הגלובאלי ואת ה-CExp שלו לסביבה. ונמשיך עם הסביבה המורחבת.**
  - **עבור LexicalAddress, אם הכתובת היא free - נחזיר את הביטוי המוגדר עבור המשתנה בסביבה, אחרת - נחזיר VarRef שלו.**

- **עבור ביטויים אטומים, נחזיר אותם עצמם**
- **עבור ביטויים מורכבים, נקרא רקורסיבית לפונקציית ההמרה עבור תתי הביטויים, ונבנה את הביטוי המורכב מחדש על בסיס תוצאת המרת תתי-הביטויים.**

<u>הערה</u>: למי שלא התייחס להצבה רק במשתנים חופשיים ירדו 2 נקודות

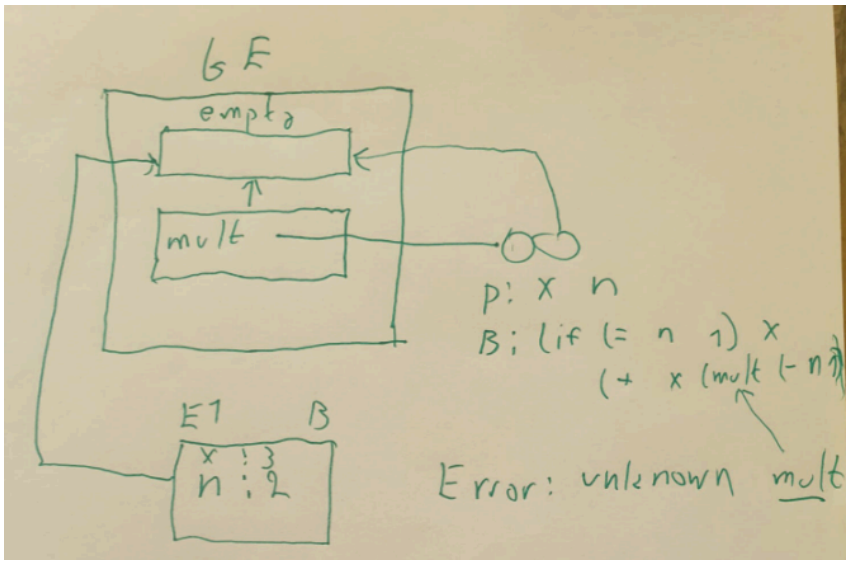
ג. ציירו דיאגרמת סביבות לכל אחת משתי התוכניות הבאות ב-L2 (בשפה זו, כזכור, אין עדיין סביבה התומכת ברקורסיות)

```
(L2

   (define mult
      (lambda (x n)
         (if (= n 1)
             x
             (+ x (mult x (- n 1))))
      )
   )

  (mult 3 2)

)
```



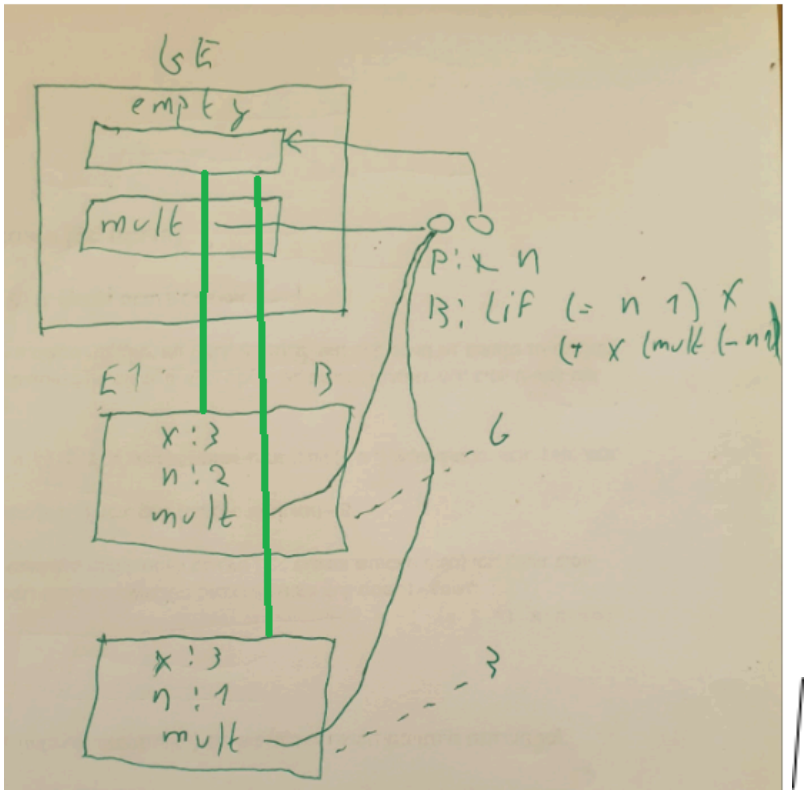

```
(L2

   (define mult
      (lambda (x n mult)
         (if (= n 1)
```

```
            x
          (+ x (mult x (- n 1) mult)))
        )
    )

  (mult 3 2 mult)

)
```



[10 נקודות]

ד. עדכנו את אלגוריתם ההמרה מ-L2 ל-L20, כך שהוא יתמוך גם בביצוע רקורסיות.
[5 נקודות]

בהינתן תוכנית ב-L2:

- **נוסיף כתובות לקסיקאליות ל-CExp ב-AST של התוכנית. כזכור, עבור VarRef של משתנים גלובאליים הכתובת הלקסיקאלית היא free.**
- **נגדיר סביבה הקושרת משתנה גלובלי ל-CExp שלו**
- **נעבור על ה-AST, החל מסביבה ריקה:**
  - **עבור DefExp, נוסיף את המשתנה הגלובאלי ואת ה-CExp שלו לסביבה. ונמשיך עם הסביבה המורחבת.**

- **עבור LexicalAddress, אם הכתובת היא free - נחזיר את הביטוי המוגדר עבור המשתנה בסביבה, אחרת - נחזיר VarRef שלו.**
- **<u>עבור AppExp שהאופרטור שלו הוא ProcExp</u>:**
  - **נוסיף ל-ProcExp פרמטר ששמו כשם ה-ProcExp**
  - **נוסיף אופרנד ל-AppExp שהוא האופרטור עצמו.**
- **עבור ביטויים אטומים, נחזיר אותם עצמם**
- **עבור ביטויים מורכבים, נקרא רקורסיבית לפונקציית ההמרה עבור תתי הביטויים, ונבנה את הביטוי המורכב מחדש על בסיס תוצאת המרת תתי-הביטויים.**

<u>הערה</u>: למי שהציע להחליף את ההפעלה של הפונקציה הרקורסיבית ב-body שלה עם ההצבה של ההפעלה, וכן הלאה לכל הקריאות הרקורסיביות ירדה נקודה אחת, בשל מסורבלות הפתרון (יצירת קוד שגודלו ביחס ישר למספר הרקורסיות, שיכול להיות גם אינסופי)

ה. האם ניתן לבצע את האלגוריתם ברמת התחביר, ללא צורך בחישוב ביטויים, או שהוא דורש חישוב של ביטויים?
[5 נקודות]

האופרטור של AppExp אינו בהכרח כבר ProcExp, ייתכן ומדובר בביטוי CExp שחישובו הוא ProcExp/Closure, כך שתהליך ההמרה דורש חישוב.

שאלה 2: טיפוסים [20 נקודות]

נרחיב את שפת הטיפוסים L5 לשפה L55 שאוכפת את הכלל הבא: **פונקציה שעשויה ליצור תופעת לוואי חייבת להצהיר על כך בטיפוס שלה**. לצורך השאלה נסתפק בתופעת הלוואי "הדפסה", כלומר פקודת `display`, ונניח שהיא היחידה הקיימת בפונקציות. על בודק הטיפוסים לוודא שכל פונקציית משתמש שיש בה שורת `display` מתויגת בהתאם בטיפוס החוזר שלה, ולהפך. אופן התיוג ייעשה כך: `prints ret-val`, כלומר הפונקציה `add` שמחברת שני מספרים, מדפיסה את הסכום ומחזירה אותו, תיראה כך:

```
(define (add : (number * number -> prints number))
    (lambda ((x : number) (y : number)) : prints number
        (let (((z : number) (+ x y)))
            (display z)
            z)))
```

המילה המיוחדת `prints` מתפרשת (בשפה טבעית) כ"הפונקציה מדפיסה *משהו*", כלומר אין חובה שמה שמודפס הוא הערך החוזר (או משהו מאותו טיפוס). הקוד הבא תקין:

```
(define (add : (number * number -> prints number))
    (lambda ((x : number) (y : number)) : prints number
        (let (((z : number) (+ x y)))
            (display #t)
            z)))
```

את מבנה הנתונים של הביטוי החדש נגדיר כך:

```
export type PrintsTExp = { tag: "PrintsTExp" }; // and predicate and make...

export type ProcTExp = { tag: "ProcTExp"; paramTEs: TExp[]; printsTE:
TExp|null; returnTE: TExp; };
export const makeProcTExp = (paramTEs: TExp[], printsTE: TExp|null, returnTE:
TExp): ProcTExp =>
    ({tag: "ProcTExp", paramTEs: paramTEs, printsTE: printsTE|null, returnTE:
returnTE});
export const isProcTExp = (x: any): x is ProcTExp => x.tag === "ProcTExp";
export const procTExpComponents = (pt: ProcTExp): TExp[] =>
    [...pt.paramTEs, pt.printsTE, pt.returnTE];

export const parseTExp = (texp: Sexp): Result<TExp> =>
    ...
    (texp === "prints") ? makeOk(makePrintsTExp()) :
    ...
```

א. עדכנו את פונקציית זיהוי ביטויי הטיפוס הפונקציונליים ב-`TExp.ts` לתמיכה בצורה החדשה:
[5 נקודות]

```
/* contents of fail messages omitted for clarity, you can do the same */
const parseCompoundTExp = (texps: Sexp[]): Result<ProcTExp> => {
    const pos = texps.indexOf('->');
    return (pos === -1)  ? makeFailure(`...`) :
           (pos === 0) ? makeFailure(`...`) :
           (pos === texps.length - 1) ? makeFailure(`...`) :
           (texps.slice(pos + 1).indexOf('->') > -1) ? makeFailure(`...`) :
           bind(parseTupleTExp(texps.slice(0, pos)), (args: TExp[]) =>
               mapv(parseTExp(texps[pos + 1]), (return1TE: TExp) =>
                   isPrintsTExp(return1TE) ?
                        (pos === texps.length - 2) ? makeFailure(`...`) :
                        mapv(parseTExp(texps[pos + 2]), (return2TE: TExp) =>
                            makeProcTExp(args, return1TE, return2TE) :
                       makeProcTExp(args, null, return1TE)));
};
```

ב. עדכנו את קוד בדיקת הטיפוסים עבור ביטויי למבדה, `typeofProc`, כך שיאכוף את ההצהרה עם הדרישה העדכנית.
[7 נקודות]

העזרו בפיסות הקוד הממוסגרות, מתוך TExp.ts ומבדיקת הטיפוסים בהתאמה:

```
export type VoidTExp = { tag: "VoidTExp" };
export const makeVoidTExp = (): VoidTExp => ({tag: "VoidTExp"});
export const isVoidTExp = (x: any): x is VoidTExp => x.tag === "VoidTExp";
```

```
export const typeofPrim = (p: PrimOp): Result<TExp> =>
    ...
    (p.op === 'display') ? parseTE('(T -> void)') :
    ...
```

```
export const allTypeofExps = (exps: List<Exp>, tenv: TEnv): Result[]<TExp> =>
    isNonEmptyList<Exp>(exps) ?
        isEmpty(rest(exps)) ? [typeofExp(first(exps), tenv)] :
        bind(typeofExp(first(exps), tenv), t =>
[t].append(allTypeofExps(rest(exps), tenv))) :
    makeFailure(`Unexpected empty list of expressions`);

export const typeofProc = (proc: ProcExp, tenv: TEnv): Result<TExp> => {
    const argsTEs = map((vd) => vd.texp, proc.args);
    const extTEnv = makeExtendTEnv(map((vd) => vd.var, proc.args), argsTEs,
tenv);
    const bodyTypes = allTypeofExps(proc.body, extTEnv);
    const constraint1 = bind(bodyTypes[-1], (body: TExp) =>
                            checkEqualType(body, proc.returnTE, proc));
```

```
    const constraint2 = any(bodyTypes.slice(0, bodyTypes.length - 1), bt =>
isVoidTExp(bt)) === proc.printsTE;
    return bind(constraint1, _ => bind(constraint2, _ =>
makeOk(makeProcTExp(argsTEs, proc.printsTE, proc.returnTE))));
};
```

ג. ההגדרה ציינה כי חובת ההצהרה חלה על פונקציה שעשויה ליצור תופעת לוואי. תארו בקווים כלליים (ללא קוד) את השינויים הנדרשים בתחביר ובבדיקת הטיפוסים (המעודכנים) במידה ונרצה לאכוף רמה נוספת: פונקציות שבהכרח מדפיסות יתויגו ב-`prints` וכאלה שאולי לא יתויגו ב-`maybe-prints`:
[8 נקודות]

```
(define (add : (number * number -> maybe-prints number))
    (lambda ((x : number) (y : number)) : maybe-prints number
        (let (((z : number) (+ x y)))
            (if (> z 0)
                ((display #t)
                 z)
                z))))
```

נצטרך לכל הפחות:
להוסיף את הביטוי המיוחד מייבי-פרינץ לתחביר, כולל פרדיקט ומייק;
לעדכן את הזיהוי שלו בתוך ביטוי מורכב של טיפוס פונקציה (כמו בסעיף א);
לאתר בתוך גוף פונקציה מבנה של תנאי ולהיכנס לתוכו כדי לוודא שרק ענף אחד מדפיס, כלומר להגדיר עוד תת-מקרה של ניתוח AST עבור ביטויי תנאי.

שאלה 3: תכנות פונקציונאלי, CPS, רשימות עצלות [30 נקודות]

א. השיבו נכון / לא נכון [2 נקודות לכל סעיף, סה״כ 10 נקודות]

א.1. אם ביטוי נמצא בעמדת זנב, אז הוא בהכרח הקריאה האחרונה בגוף הפונקציה.
תשובה: **נכון**

א.2. עמדת זנב, **באינטרפרטר התומך באופטימיזציית זנב**, מבטיחה שקריאה לפונקציה לא תוסיף פריים חדש למחסנית הקריאות.
תשובה: **נכון**

א.3. כתיבה של פונקציה בסגנון CPS מבטיחה שהקריאה תמיד תהיה בזנב.
תשובה: **נכון**

א.4. הקריאה (f (g x)) נמצאת בעמדת זנב אם f היא הקריאה האחרונה.
תשובה: **לא נכון**

א.5. עמדת זנב רלוונטית רק בשפות פונקציונליות.
תשובה: **לא נכון**

ב. רשימות עצלות ו-CPS
[12 נקודות]
ב.1. [6 נקודות] הפונקציה take שימושית מאוד בעולם הרשימות העצלות. היא מקבלת רשימה עצלה ומס׳ איברים, ומחשבת רשימה רגילה המכילה את מס׳ האיברים הראשונים מתוך הרשימה העצלה. להלן הקוד שלה:

```
;; [ Lzl[T] * Number -> List[T] ]
(define take
  (λ (lzl n)
     (if (or (= n 0) (empty-lzl? lzl))
         empty
         (cons (head lzl)
           (take (tail lzl) (- n 1))))))
```

כתבו גרסה בסגנון CPS עבור פונקציה זו:

```
;; [ Lzl[T1] * Number * [List[T1] -> T2] -> T2 ]
(define take$
  (λ (lzl n cont)
     (if (or (= n 0) (empty-lzl? lzl))
         (cont empty)
         (take$ (tail lzl)
                (- n 1)
                (λ (res-tail)
                     (cont (cons (head lzl) res-tail)))))))
```

ב.2. [8 נקודות] הוכיחו כי שתי הפונקציות שקולות-CPS
צריך להוכיח כי שתי הפונקציות שקולות CPS כלומר לכל מספר טבעי n, לכל רשימה עצלה lzl, ולכל פונקציה cont, מתקיים:

```
(take$ lzl n cont) = (cont (take lzl n))
```

נוכיח זאת באמצעות שימוש באינדוקציה.
נשים לב כי יש לנו שני מקרי בסיס:
בסיס 1: n=0

```
a-e[ (take$ lzl 0 cont) ⇒ (cont empty-lzl)
a-e[ (cont (take lzl 0) ⇒ (cont empty-lzl)
```

מתקיים בסיס לאינדוקציה.
הנחות האינדוקציה:
הטענה מתקיימת עבור n=k∈ N, ועבור רשימה עצלה באורך l, כלומר:

```
(take$ lzl k cont) = (cont (take$ lzl k cont))
```

צעד האינדוקציה: יהא n=k+1,k∈N, ו-lzl רשימה לא ריקה:
(לצורך נוחות וקיצור נסמן: (head lzl) ב-h ו-(tail lzl) ב-t)

```
a-e [ (cont (take lzl n) ] ==>*
a-e [ (cont (cons h (take t (- n 1) ))) ] =

a-e [ (take$ lzl n cont) ] ==>*
a-e [ (take$ t (- n 1) (λ (res-tail) (cont (cons h res-tail)))) ]
```

מהנחת האינדוקציה לכל cont מתקיים:

```
a-e [ (take$ t (- n 1) (λ (res-tail) (cont (cons h res-tail)))) ] =
a-e [ ((λ (res-tail) (cont (cons h res-tail))) (take t (- n 1))) ] =
a-e [ (cont (cons h (take t (- n 1)))) ] =
a-e [ (cont (take lzl n)) ]
```

■

ג. תכנות פונקציונלי ו- Currying פונקציונלי [6 נקודות]
השלימו את הקוד עבור הפונקציה make-cond-accumulator אשר מקבלת כפרמטרים פונקציית צבירה/מיזוג (למשל + או *) וערך נייטרלי (0 או 1 בהתאמה למשל), ומחזירה פונקציה נוספת המקבלת פונקציית פרדיקט ורשימה ומבצעת את הצבירה רק על איברים העונים על התנאי בפרדיקט. ראו דוגמאות בחתימת הפונקציה:

```
;;[ [T1 * T2 -> T2] * T2 -> [[T1 -> Boolean] * List[T1] -> T2] ]
;; Examples:
;;  ((make-cond-accumulator + 0) (λ (x) (< x 5)) '(1 2 5 6 3))
;;  => 6
```

```
;;  ((make-cond-accumulator cons '()) (λ (x) (< x 5)) '(1 2 5 6 3))
;;  => '(1 2 3)
(define make-cond-accumulator
  (λ (combine neutral)
     (λ (pred lst)
     (cond
       ((null? lst) neutral)
       ((pred (car lst))
       (combine
          (car lst)
          ((make-cond-accumulator combine neutral)
          pred
          (cdr lst))))
       (else
         ((make-cond-accumulator combine neutral)
          pred
          (cdr lst)))))))
```

שאלה 4: תכנות לוגי [20 נקודות]

נגדיר עץ בינארי ע"פ החוק/יחס הבא:

```
binary_tree(void).
binary_tree(tree(Element,Left,Right)) :-
   binary_tree(Left), binary_tree(Right).
```

א. האם ניתן להגדיר עץ בינארי ללא שימוש בפנקטורים? נמקו.
[2 נקודות]

לא. הדרך היחידה לעשות זאת במקרה זה היתה על ידי חוקים, משהו כמו:

```
binary_tree(void).
binary_tree(Element,Left,Right) :-
   binary_tree(Left), binary_tree(Right).
```

אך במקרה כזה העץ יכול להיות רק בעל קודקוד אחד, כי אלו שני פרדיקטים בעלי אריטי שונה.

ב. ממשו את הפרוצדורה `tree_to_list` הנותנת ערך אמת לעץ ולרשימה, אם הרשימה כוללת את האלמנטים של העץ בסדר in_order (קודם האלמנטים של תת-העץ השמאלי, אחר כך האלמנט של העץ, ואחר כך האלמנטים של תת-העץ הימני). ניתן להשתמש ב-`append` שנלמדה בכיתה.

```
?-tree_to_list(tree(a,tree(b,void,void),tree(c,void,void)),L)
L = [b,a,c]

?-tree_to_list(tree(a,tree(b,tree(d,void,void),void),tree(c,void,tree
(e,void,void))),L)
L = [d,b,a,c,e]
```

```
% Signature: tree_to_list(Tree,List)/2
% Purpose: List is a list of the Elements in Tree (inorder)
tree_to_list(void,[]).
tree_to_list(tree(Element,Left,Right),Lst):-
    tree_to_list(Left,Lefts),
    tree_to_list(Right,Rights),
    append(Lefts,[Element],L1),
    append(L1,Rights,Lst).
```

[8 נקודות]

ג. ציירו את עץ ההוכחה עבור השאילתא הבאה, וציינו האם העץ הוא סופי או אינסופי והאם הוא עץ הצלחה או כישלון:

```
person(danny,location(bs)).     %1
person(dinna,location(tlv)).    %2
person(ruth,location(jrs)).     %3

course(spl,bgu,location(bs)).     %4
course(ppl,tau,location(tlv)).      %5
course(algo,heb,location(jrs)).     %6

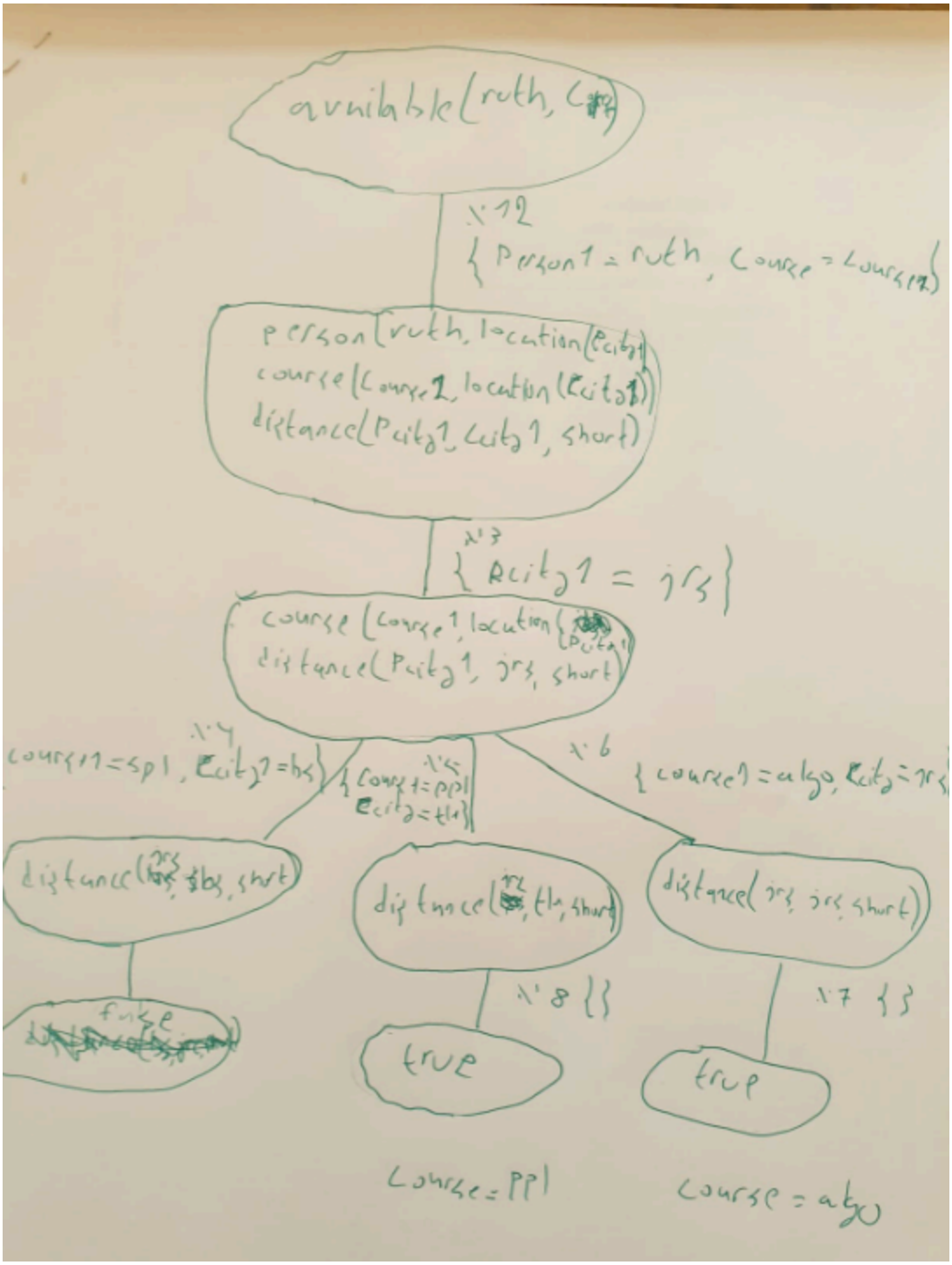
distance(jrs,bs,long).              %7
distance(jrs,tlv,short).              %8
distance(jrs,jrs,short).            %9

available(Person,Course):-                %10
   person(Person,location(PCity)),
   course(Course,_Univ,location(CCity)),
   distance(PCity,CCity,short).

?-available(ruth,Course)
```

[10 נקודות]

עץ הצלחה סופי:


available(ruth, C
λ12
{Person1 = ruth, Course = Course12}
person(ruth, location(City1
course(Course1, location(City1))
distance(City1, City1, short)
λ7
{City1 = jrs}
course(Course1, location(
distance(City1, jrs, short
λ4
Course11 = spl, City2 = bks}
λ5
{Course1 = ppl
City2 = tls}
λ6
{course1 = algo, City2 = jrs
distance(jrs, bks, short)
distance(jrs, tls, short)
distance(jrs, jrs, short)
λ8 {}
λ7 {}
true
true
Course = ppl
Course = algo